

Permissioned vs. Public Blockchain for Audit Log Management: A Comparative Analysis of Integrity, Performance, and Governance

João Guedes de Brito

Salvador, Bahia, Brazil

joaoguedesdebrito@gmail.com

Abstract

Audit log integrity is a critical requirement in security-sensitive systems, where tampering or unauthorized modification of event records can compromise forensic investigations, regulatory compliance, and organizational trust. Blockchain technology offers a decentralized and immutable alternative to traditional centralized log storage. However, the choice between permissioned and public blockchain architectures involves significant trade-offs across dimensions such as data integrity guarantees, transaction throughput, latency, governance models, privacy, and operational cost. This paper presents a systematic comparative analysis of permissioned (Hyperledger Fabric) and public (Ethereum) blockchain implementations for audit log management in web application authentication systems. We design and implement identical audit logging pipelines on both platforms, evaluating them across seven criteria: immutability guarantees, write throughput, transaction confirmation latency, data privacy and GDPR/LGPD compliance, governance flexibility, operational cost, and scalability. Experimental results demonstrate that Hyperledger Fabric achieves 3,500 transactions per second (TPS) with sub-second finality ($<0.5s$), while Ethereum provides 15–30 TPS with 12-block confirmation latency ($\approx 180s$). Both platforms achieved 100% integrity verification in tamper detection tests. The analysis reveals that permissioned blockchains are better suited for enterprise audit log management due to superior throughput, deterministic finality, and built-in

access control, while public blockchains offer stronger censorship resistance and decentralization guarantees suitable for cross-organizational or trustless environments. We propose a decision framework to guide practitioners in selecting the appropriate blockchain architecture based on their audit log requirements.

Keywords: Blockchain, Audit Logs, Permissioned Blockchain, Public Blockchain, Hyperledger Fabric, Ethereum, Data Integrity, Log Management, LGPD, GDPR

1 Introduction

Audit logs constitute a fundamental pillar of information security, providing forensic evidence of system events, user activities, and security incidents. In regulated environments—such as financial systems, healthcare platforms, and government applications—the integrity, availability, and non-repudiation of audit logs are not merely desirable properties but legal requirements mandated by regulations including the European General Data Protection Regulation (GDPR) [5] and Brazil’s Lei Geral de Proteção de Dados (LGPD) [2].

Traditional audit log management relies on centralized databases, where log entries are stored in relational or document-oriented systems under the control of system administrators. This centralized approach presents inherent vulnerabilities: administrators with privileged access can modify, delete, or forge log entries without leaving detectable traces [10]. Such tampering undermines the evidentiary value of logs and compromises the entire chain of trust in security investigations.

Blockchain technology offers a compelling alternative by providing structurally guaranteed immutability through cryptographic hash chaining, consensus mechanisms, and distributed replication [7]. Once a transaction is confirmed and appended to the chain, retroactive modification becomes computationally infeasible without invalidating all subsequent blocks. This property makes blockchain architecturally suited for audit log management, where the primary requirement is ensuring that recorded events cannot be tampered with after the fact.

However, blockchain is not a monolithic technology. Two fundamentally different architectural paradigms exist:

- **Public blockchains** (e.g., Ethereum) operate as open, permissionless networks where any participant can read, write, and validate transactions. They achieve consensus through mechanisms such as Proof-of-Work (PoW) or Proof-of-Stake (PoS),

prioritizing decentralization and censorship resistance at the cost of throughput and latency [4].

- **Permissioned blockchains** (e.g., Hyperledger Fabric) restrict participation to authorized entities, employing consensus protocols such as Raft or Byzantine Fault Tolerance (BFT) that enable higher throughput and deterministic finality, at the cost of reduced decentralization [6].

Despite the growing body of literature on blockchain applications, comparative studies specifically targeting audit log management remain scarce. Existing comparisons typically focus on cryptocurrency performance or supply chain use cases, without addressing the unique requirements of security-critical log storage: high write throughput for continuous event logging, low latency for real-time auditing, strong privacy controls for personally identifiable information (PII), and verifiable integrity guarantees for forensic investigations.

This paper addresses this gap by presenting a systematic comparative analysis of Hyperledger Fabric and Ethereum for audit log management in web application authentication systems. We design identical logging pipelines on both platforms and evaluate them across seven dimensions: (1) immutability guarantees, (2) write throughput, (3) transaction confirmation latency, (4) data privacy and regulatory compliance, (5) governance flexibility, (6) operational cost, and (7) scalability. Based on the experimental results, we propose a decision framework to guide practitioners in selecting the appropriate blockchain architecture for their specific audit log requirements.

The main contributions of this paper are:

1. A systematic comparative framework for evaluating blockchain architectures in the context of audit log management;
2. Empirical performance measurements of Hyperledger Fabric and Ethereum under identical audit logging workloads;
3. A practical decision framework for blockchain architecture selection based on audit log requirements;
4. An analysis of privacy and regulatory compliance implications (GDPR/LGPD) for each architecture.

The remainder of this paper is organized as follows: Section 2 reviews related work; Section 3 presents background concepts; Section 4 describes the experimental architecture; Section 5 details the evaluation methodology; Section 6 presents and discusses results; and Section 8 concludes with recommendations and future work.

2 Related Work

Blockchain-based audit logging has attracted increasing attention in the literature. Cucurull and Puiggali [3] proposed using Bitcoin’s blockchain for tamper-evident logging, demonstrating that anchoring log hashes to a public blockchain provides independently verifiable integrity proofs. However, their approach was limited to periodic batch anchoring due to Bitcoin’s throughput constraints.

Ahmad et al. [1] designed BlockAudit, a blockchain-based audit trail system for cloud services using Hyperledger Fabric, achieving throughput of 2,000 TPS for audit event recording. Their work focused exclusively on permissioned architectures without comparing against public alternatives.

Pourmajidi and Miranskyy [13] proposed Logchain, a blockchain-based log management framework, evaluating both Ethereum and Hyperledger for cloud audit trails. Their comparison, however, was conducted on early versions of both platforms and did not address privacy implications or regulatory compliance.

In the broader blockchain comparison literature, Pongnumkul et al. [12] performed one of the first empirical comparisons between Ethereum and Hyperledger Fabric, measuring execution time, latency, and throughput. Their seminal work established the baseline understanding that permissioned blockchains significantly outperform public ones in controlled environments, but did not contextualize findings for audit log use cases.

More recently, Nasir et al. [8] surveyed blockchain applications in cybersecurity, identifying audit trail management as a key use case while noting the absence of systematic comparisons tailored to log integrity requirements.

The NIST Blockchain Technology Overview (NISTIR 8202) [9] provides an authoritative taxonomy of blockchain architectures, distinguishing permissioned from permissionless networks and discussing their applicability to different trust models. The report recommends permissioned blockchains for enterprise environments requiring governance and identity management.

This work distinguishes itself by providing a comprehensive, multi-dimensional comparison specifically designed for the audit log management use case, incorporating contem-

porary platform versions, privacy analysis under GDPR/LGPD, and a practical decision framework for practitioners.

3 Background

3.1 Blockchain Fundamentals

A blockchain is a distributed data structure consisting of an ordered sequence of blocks, each containing a set of validated transactions and a cryptographic hash of the preceding block [7]. This hash-chain structure ensures that any modification to a historical block invalidates all subsequent hashes, providing a tamper-evident property that is structurally guaranteed rather than policy-dependent.

Consensus mechanisms determine how network participants agree on the validity and ordering of transactions. Different mechanisms represent different trade-offs between decentralization, throughput, finality, and fault tolerance.

3.2 Ethereum (Public Blockchain)

Ethereum is a public, permissionless blockchain that transitioned from Proof-of-Work to Proof-of-Stake consensus in September 2022 (“The Merge”) [4]. Key characteristics relevant to audit logging include:

- **Consensus:** Proof-of-Stake with 32 ETH minimum validator stake;
- **Throughput:** 15–30 TPS on the main network (Layer 1);
- **Finality:** Probabilistic; typically 12 blocks (≈ 180 seconds) for practical finality;
- **Data visibility:** All transactions are publicly visible on-chain;
- **Cost:** Variable gas fees per transaction;
- **Smart contracts:** Turing-complete (Solidity/EVM).

3.3 Hyperledger Fabric (Permissioned Blockchain)

Hyperledger Fabric is an enterprise-grade, permissioned blockchain framework hosted by the Linux Foundation [6]. Key characteristics include:

- **Consensus:** Pluggable (Raft for crash fault tolerance; BFT available);
- **Throughput:** 3,000–20,000 TPS depending on configuration;
- **Finality:** Deterministic; transactions are final upon block commit ($<0.5s$);
- **Data visibility:** Private data collections and channels enable selective disclosure;
- **Cost:** No per-transaction fees; infrastructure costs only;
- **Smart contracts:** Chaincode (Go, Java, Node.js).

3.4 Audit Log Requirements

Based on NIST SP 800-92 (Guide to Computer Security Log Management) [11] and regulatory requirements (GDPR, LGPD), we identify the following essential requirements for audit log systems:

1. **Integrity:** Log entries must be tamper-evident and resistant to unauthorized modification;
2. **Non-repudiation:** The origin and content of log entries must be verifiable;
3. **Availability:** Logs must be accessible for forensic analysis and compliance audits;
4. **Confidentiality:** PII within logs must be protected (GDPR Art. 5, LGPD Art. 6);
5. **Timeliness:** Logging must occur with minimal delay to capture real-time events;
6. **Scalability:** The system must handle high volumes of continuous event streams;
7. **Compliance:** The storage mechanism must support regulatory requirements including the right to erasure (GDPR Art. 17) and data minimization.

4 Experimental Architecture

4.1 System Design

To ensure a fair comparison, we designed an identical audit log pipeline deployed on both blockchain platforms. The architecture consists of three layers:

1. **Event Generation Layer:** A Spring Boot application simulating authentication events (login success, login failure, MFA triggered, access blocked) with contextual metadata (user ID hash, IP address, device fingerprint, timestamp, risk score).
2. **Log Processing Layer:** A middleware service that receives events, applies data transformations (PII hashing, field normalization), and prepares blockchain transactions.
3. **Blockchain Storage Layer:** Parallel deployment on both Ethereum (Sepolia test-net / local Ganache) and Hyperledger Fabric (2-org network with Raft consensus).

4.2 Audit Log Data Model

Each audit log entry is represented as a structured transaction containing the fields described in Table 1.

Table 1: Audit log data model for blockchain transactions

Field	Type	Description
eventId	String	Unique event identifier (UUID)
eventType	Enum	LOGIN_SUCCESS, LOGIN_FAILURE, MFA_REQUIRED, AC- CESS_BLOCKED
userIdHash	String	SHA-256 hash of user ID (anonymized)
ipAddress	String	Source IP address
deviceHash	String	SHA-256 hash of device fingerprint
riskScore	Float	AI-computed risk score [0, 1]
decision	Enum	ALLOW, MONITOR, RE- QUIRE_MFA, BLOCK
timestamp	Long	Unix timestamp (milliseconds)
payloadHash	String	SHA-256 hash of the complete event payload

Personal identifiers are pre-hashed before blockchain submission, ensuring that no PII is stored on-chain—a critical requirement for GDPR/LGPD compliance, particularly re-

garding the right to erasure (Art. 17 GDPR / Art. 18 LGPD), since immutable blockchain records cannot be deleted.

4.3 Ethereum Implementation

The Ethereum implementation utilizes a Solidity smart contract deployed on both a local Ganache instance (for controlled benchmarking) and the Sepolia public testnet (for realistic network conditions). The contract exposes two primary functions:

- `recordEvent(bytes32 eventHash, uint8 eventType, uint256 timestamp)`: Stores the hash and metadata of an audit event;
- `verifyEvent(bytes32 eventHash)`: Returns the stored record for integrity verification.

Integration is performed via Web3j 4.10.3, a Java library for Ethereum interaction.

4.4 Hyperledger Fabric Implementation

The Hyperledger Fabric network consists of two organizations (Org1, Org2), each with two peers, one ordering node (Raft consensus), and one Certificate Authority (CA) per organization. The chaincode (Go) implements equivalent functions:

- `RecordEvent(eventHash, eventType, timestamp)`: Creates a key-value entry in the world state;
- `VerifyEvent(eventHash)`: Queries the world state and returns the event record;
- `GetEventHistory(eventHash)`: Returns the full modification history from the ledger.

Private data collections are configured to restrict raw event data to authorized organizations, while hashed summaries are shared across the channel.

5 Evaluation Methodology

5.1 Evaluation Criteria

We evaluate both blockchain implementations across seven dimensions, as summarized in Table 2.

Table 2: Evaluation criteria for blockchain audit log comparison

#	Criterion	Measurement
C1	Immutability	Tamper detection success rate (%)
C2	Write Throughput	Transactions per second (TPS)
C3	Confirmation Latency	Time to finality (seconds)
C4	Privacy & Compliance	PII exposure, right to erasure support
C5	Governance	Access control, identity management
C6	Operational Cost	Per-transaction and infrastructure cost
C7	Scalability	TPS degradation under increasing load

5.2 Workload Design

The benchmark workload simulates a production authentication system generating audit events at varying rates:

- **Low load:** 10 events/second (small web application);
- **Medium load:** 100 events/second (enterprise application);
- **High load:** 1,000 events/second (high-traffic platform);
- **Stress load:** 5,000 events/second (peak/stress testing).

Each test was executed for 5 minutes per load level, with 3 repetitions for statistical significance. Event payloads followed the data model in Table 1, with randomized but realistic values.

5.3 Tamper Detection Tests

To evaluate immutability (C1), we designed three tamper scenarios:

1. **T1 – Record modification:** Attempt to alter an existing log entry’s risk score and decision fields;

2. **T2 – Record deletion:** Attempt to remove a specific audit event from the ledger;
3. **T3 – Backdating:** Attempt to insert a fabricated event with a past timestamp into the historical chain.

For each scenario, the verification function was called before and after the tamper attempt to determine detection success.

5.4 Environment

All experiments were conducted on identical hardware: Intel Core i7-12700K, 32GB RAM, 1TB NVMe SSD, running Docker containers on Ubuntu 22.04 LTS. Ethereum was tested on both local Ganache (isolated) and Sepolia testnet (realistic). Hyperledger Fabric 2.5 was deployed with the configuration described in Section 4.

6 Results and Discussion

6.1 C1: Immutability Guarantees

Both platforms achieved 100% tamper detection across all three test scenarios (T1, T2, T3), as shown in Table 3.

Table 3: Tamper detection results across three scenarios

Platform	T1 (Modify)	T2 (Delete)	T3 (Backdate)
Ethereum (Ganache)	100%	100%	100%
Ethereum (Sepolia)	100%	100%	100%
Hyperledger Fabric	100%	100%	100%

Both architectures provide structurally guaranteed immutability through hash chaining. The key difference lies in the trust model: Ethereum’s immutability is backed by thousands of independent validators (trustless), while Fabric’s is backed by the consortium of known organizations (trusted consortium). For intra-organizational audit logging, Fabric’s model is sufficient; for cross-organizational or regulatory scenarios requiring independent verification, Ethereum’s public verifiability is advantageous.

6.2 C2: Write Throughput

Table 4 presents the measured write throughput under each load level.

Table 4: Write throughput (TPS) under varying load levels				
Platform	Low (10)	Med (100)	High (1K)	Stress (5K)
Ethereum (Ganache)	10	100	285	312
Ethereum (Sepolia)	10	15	15	15
Hyperledger Fabric	10	100	1,000	3,500

Hyperledger Fabric sustained the target throughput at all load levels, reaching 3,500 TPS under stress. Ethereum’s local instance (Ganache) plateaued at ≈ 312 TPS due to single-node limitations, while the Sepolia testnet was constrained to ≈ 15 TPS by network-level block gas limits. This represents a critical limitation for production audit logging: a system generating 100+ events/second would experience event queuing and potential data loss on a public Ethereum network.

6.3 C3: Confirmation Latency

Table 5 shows the transaction confirmation latency for each platform.

Table 5: Transaction confirmation latency		
Platform	Avg. Latency	Finality Type
Ethereum (Ganache)	<1s (instant mine)	Simulated
Ethereum (Sepolia)	≈ 180 s (12 blocks)	Probabilistic
Hyperledger Fabric	<0.5s	Deterministic

Hyperledger Fabric provides deterministic finality in sub-second time, meaning that once a transaction is committed, it is immediately and irrevocably final. Ethereum’s probabilistic finality requires waiting for multiple block confirmations to achieve practical immutability; the standard 12-block recommendation translates to approximately 3 minutes—a significant delay for real-time audit logging scenarios where immediate verifiability is required.

6.4 C4: Privacy and Regulatory Compliance

Table 6 summarizes the privacy characteristics of each platform.

Table 6: Privacy and compliance comparison

Feature	Ethereum	Hyperledger Fabric
On-chain data visibility	Public (all nodes)	Restricted (channels/PDC)
PII protection	Hash-only strategy	Hash + private data collections
Right to erasure (Art.17/18)	Not natively supported	Partial (PDC expiration)
Access control	None (permissionless)	MSP + channel policies
Data minimization	Manual (off-chain)	Built-in (PDC)

Both platforms require a “hash-only” strategy to avoid storing PII on-chain. Hyperledger Fabric offers a significant advantage through Private Data Collections (PDC), which allow raw audit data to be shared only among authorized organizations and can be configured with automatic expiration—providing a partial solution to the right-to-erasure requirement. Ethereum has no native mechanism for data restriction; all on-chain data is globally visible, making the hash-only approach mandatory for any scenario involving personal data.

6.5 C5: Governance

Ethereum operates as a fully decentralized network with no central authority controlling participation or transaction validation. While this provides censorship resistance, it also means that no single organization can enforce audit policies, revoke access, or manage participant identities.

Hyperledger Fabric provides comprehensive governance through its Membership Service Provider (MSP), channel-based access control, and endorsement policies. Organizations can define precisely which entities are authorized to submit, read, or validate audit log transactions. This aligns with enterprise security requirements where audit access must be controlled and auditable itself.

6.6 C6: Operational Cost

Table 7: Operational cost comparison

Cost Factor	Ethereum	Hyperledger Fabric
Per-transaction fee	Variable (gas)	None
Estimated cost (1M tx)	\$500–\$5,000+	\$0 (infrastructure only)
Infrastructure	Node or provider	Self-hosted cluster
Monthly infrastructure	\$50–\$200 (Infura/Alchemy)	\$100–\$500 (cloud VMs)

Ethereum incurs per-transaction gas fees that are highly variable and can spike during network congestion. For high-volume audit logging (millions of events), these costs become prohibitive. Hyperledger Fabric eliminates per-transaction fees entirely; the only costs are infrastructure (compute, storage, networking), which are predictable and scale linearly.

6.7 C7: Scalability

Hyperledger Fabric demonstrated near-linear scalability up to 3,500 TPS in our configuration, with the potential for higher throughput through horizontal scaling (additional peers, parallel channels). Ethereum Layer 1 is fundamentally limited to $\approx 15\text{--}30$ TPS, though Layer 2 solutions (rollups, sidechains) can increase effective throughput to thousands of TPS at the cost of additional architectural complexity and trust assumptions.

6.8 Summary of Comparative Results

Table 8 provides a consolidated comparison across all seven evaluation criteria.

Table 8: Consolidated comparison: Ethereum vs. Hyperledger Fabric for audit logs

Criterion	Ethereum	Hyperledger Fabric	Winner
C1: Immutability	100%	100%	Tie
C2: Throughput	15–30 TPS	3,500 TPS	Fabric
C3: Latency	≈ 180 s	< 0.5 s	Fabric
C4: Privacy	Hash-only	PDC + channels	Fabric
C5: Governance	Decentralized	MSP + policies	Context-dependent
C6: Cost	Variable (gas)	Infrastructure only	Fabric
C7: Scalability	Limited (L1)	Near-linear	Fabric

6.9 Decision Framework

Based on our findings, we propose the following decision framework for practitioners:

Choose Hyperledger Fabric when:

- The audit system operates within a single organization or trusted consortium;
- High throughput (>100 events/second) is required;
- Real-time finality is necessary for immediate log verification;
- GDPR/LGPD compliance requires data access control and potential erasure;
- Operational cost must be predictable and minimized.

Choose Ethereum (public) when:

- Audit logs must be independently verifiable by untrusted third parties;
- Censorship resistance is a requirement (no single entity can suppress logs);
- Cross-organizational auditing without a trusted intermediary is needed;
- Periodic anchoring (batch hashing) is acceptable rather than per-event logging;
- The volume of events is low (<15 events/second).

Consider a hybrid approach when:

- The system requires both high throughput and public verifiability;
- In this model, Hyperledger Fabric handles real-time event logging, while periodic Merkle root anchoring to Ethereum provides an independently verifiable integrity checkpoint.

7 Threats to Validity

Internal validity. The Ethereum benchmarks on Ganache represent idealized conditions not reflective of mainnet performance. We mitigate this by also testing on the Sepolia public testnet. Hyperledger Fabric’s performance is configuration-dependent; our 2-organization setup may not represent larger consortium deployments.

External validity. Our workload simulates authentication audit events with a specific data model. Other audit log use cases (e.g., financial transactions, healthcare records) may have different characteristics that affect the comparative results.

Construct validity. The seven evaluation criteria, while comprehensive for audit log management, may not capture all dimensions relevant to specific deployments (e.g., interoperability, developer ecosystem maturity).

8 Conclusion and Future Work

This paper presented a systematic comparative analysis of permissioned (Hyperledger Fabric) and public (Ethereum) blockchain architectures for audit log management in web application authentication systems. Through identical experimental pipelines evaluated across seven dimensions, we demonstrate that:

1. Both architectures provide equivalent immutability guarantees (100% tamper detection) through cryptographic hash chaining;
2. Hyperledger Fabric outperforms Ethereum by two orders of magnitude in throughput (3,500 vs. 15–30 TPS) and three orders of magnitude in confirmation latency (<0.5s vs. ≈ 180 s);
3. Permissioned blockchains offer superior privacy controls, governance capabilities, and cost predictability for enterprise audit logging;
4. Public blockchains provide unique advantages in censorship resistance and trustless verifiability for cross-organizational scenarios.

The proposed decision framework provides practitioners with actionable guidance for blockchain architecture selection based on their specific requirements. For the majority of enterprise audit log management scenarios, Hyperledger Fabric represents the more suitable choice. However, hybrid approaches combining permissioned real-time logging with periodic public anchoring offer a compelling middle ground.

Future work includes: (i) evaluation of Layer 2 Ethereum solutions (Optimistic/ZK Rollups) for audit logging; (ii) performance analysis with larger Fabric network configurations (10+ organizations); (iii) implementation and evaluation of the proposed hybrid anchoring approach; (iv) cost-benefit analysis over extended deployment periods; and (v) formal verification of the smart contract and chaincode implementations for audit log integrity.

References

- [1] S. Ahmad, R. Dar, Blockchain-based audit trail systems for cloud computing: A systematic review, *Journal of Cloud Computing* 8 (2019) 1–18.
- [2] Brasil, Lei nº 13.709, de 14 de agosto de 2018, Lei Geral de Proteção de Dados Pessoais (LGPD), Diário Oficial da União, Brasília, DF, 2018.
- [3] J. Cucurull, J. Puiggalí, Distributed immutabilization of secure logs, in: *Proc. Int. Workshop on Security and Trust Management*, Springer, 2016, pp. 122–137.
- [4] Ethereum Foundation, Ethereum Documentation: Proof-of-Stake, 2024. Available at: <https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/>.
- [5] European Parliament, Regulation (EU) 2016/679 – General Data Protection Regulation (GDPR), Official Journal of the European Union, 2016.
- [6] Hyperledger Foundation, Hyperledger Fabric Documentation, v2.5, Linux Foundation, 2024.
- [7] S. Nakamoto, Bitcoin: A Peer-to-Peer Electronic Cash System, 2008.
- [8] M.H. Nasir, J. Arshad, M.M. Khan, M. Fatima, K. Salah, R. Jayaraman, Scalable blockchains – A systematic review, *Future Generation Computer Systems* 126 (2022) 136–162.
- [9] NIST, Blockchain Technology Overview, NISTIR 8202, Gaithersburg, 2018.
- [10] NIST, Security and Privacy Controls for Information Systems and Organizations, SP 800-53 Rev. 5, Gaithersburg, 2020.
- [11] NIST, Guide to Computer Security Log Management, SP 800-92, Gaithersburg, 2006.
- [12] S. Pongnumkul, C. Siripanpornchana, S. Thajchayapong, Performance analysis of private blockchain platforms in varying workloads, in: *Proc. 26th Int. Conf. on Computer Communication and Networks (ICCCN)*, IEEE, 2017, pp. 1–6.
- [13] W. Pourmajidi, A. Miransky, Logchain: Blockchain-assisted log storage, in: *Proc. IEEE 11th Int. Conf. on Cloud Computing (CLOUD)*, IEEE, 2018, pp. 978–982.

Declaration of Competing Interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

The experimental data and source code used in this study are available from the author upon reasonable request.

CRedit Author Statement

João Guedes de Brito: Conceptualization, Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Writing – Original Draft, Writing – Review & Editing, Visualization.

Acknowledgements

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of Generative AI in Scientific Writing

The author declares that generative AI tools were used exclusively for linguistic review and text structuring assistance during the preparation of this manuscript. All technical content, experimental design, implementation, data analysis, and scientific conclusions are the sole responsibility of the author. The author reviewed and edited all AI-assisted content and takes full responsibility for the final publication.